

Remote ATM Access Protocol Technical Project File

1. Project Purpose and Context

1.1. Social Need and Target Audience

This project aims to eliminate physical barriers encountered by individuals with limited mobility (disabled, elderly, or bedridden) in accessing financial services. In today's digital world, access to cash constitutes a logistical burden and a physical barrier for this sensitive user group. The core focus of the project is to enable the user to project their savings authority over their own bank account onto physical ATM processes without leaving the safe environment of their home. The sustainability of this social solution and user security are directly related to the technical robustness of the system and full compliance with banking security protocols.

1.2. Core Solution Approach

The system merges the user's interface at home with the physical ATM interaction in the field via a digital bridge. While a remote Android instance running on a central ARM64 server provides access to the user's banking application, the physical presence of the field representative at the ATM is virtualized using camera and GPS simulation techniques. This approach solves physical location and visual data requirements through a central automation channel without violating banking security.

2. System Architecture and Infrastructure Choice

2.1. ARM64 Architecture Requirement

Banking applications use local C/C++ libraries, usually obfuscated, called via JNI (Java Native Interface) due to high-level security and encryption requirements. Binary translation layers such as libhoudini or binfmt_misc used to run these libraries on standard servers with x86_64 architecture cannot correctly process encrypted memory instructions and lead to SIGSEGV (Segmentation Fault) errors caused by "Architecture Conflict." To prevent memory access violations and ensure stable operation of banking libraries, the use of Native ARM64 (Oracle Ampere A1 or AWS Graviton) infrastructure is a technical requirement.

2.2. Android Operating Environment

Docker-based Redroid (Remote Android) container architecture is used on Ubuntu 22.04 LTS. Preferring Native ARM64 images instead of emulation allows CPU and GPU commands to be transmitted directly to the hardware via the "Kernel-level passthrough" method. This architecture provides the hardware performance required by banking applications by minimizing virtualization latencies.

3. Step-by-Step Operational Workflow

3.1. User Login and Request Creation

The user logs into the YKB (Yapı Kredi Bankası) application with their own credentials via the remote Android instance. Following the login process, a "Withdraw Cash via QR" request is created in the application interface. At this stage, the application starts waiting for camera and location data.

3.2. Pool Management and Field Synchronization

Transaction data is held in a virtual pool operating with a central "State-machine" logic. This pool matches each user session ID with the data flow of the relevant operator currently in the field. When the field operator reaches the ATM point, the QR code stream captured by the operator is directed to the correct Android instance via this message queue.

3.3. QR Code Injection and Verification

The QR code image transmitted instantly by the operator in the field creates a virtual video device node (device node) such as `/dev/video0` via V4L2Loopback (Video4Linux2 Loopback) on the server side. The image stream injected into this virtual device using FFmpeg is perceived by the Android operating system as a physical hardware peripheral (hardware camera). The bank application verifies the QR code within seconds via this virtual camera.

3.4. Location Simulation and Transaction Approval

To complete the transaction, the device's GPS coordinates must overlap with the ATM location. To bypass advanced security checks, coordinates are injected directly into the "Location Manager" service via LSPosed modules. Since this method manipulates the location without activating the "Allow Mock Locations" setting under "Developer Options," the application's security barriers are not triggered. Following location and QR verification, the ATM performs the physical cash dispense.

4. Technical Challenges and Overcoming Security Barriers

4.1. Root and Emulator Detection

Modern banking applications check the system's root directory access, bootloader lock, and hardware signature at runtime. If any trace of emulation or Root authority is detected, the application stops execution.

4.2. Bypass Strategies

"Runtime Hooking" and "Method Interception" techniques are applied to overcome firewalls:

- **Zygisk and Shamiko:** Used to hide Root authorities. Thanks to Shamiko's "Whitelist"

mode, the environment seen only by the bank application is completely cleaned.

- **Hide My Applist:** Hides the package names of Magisk, LSPosed, or other technical tools on the device from application queries.
- **LSPosed Framework:** The basic hooking infrastructure used to manipulate system calls and intercept methods at runtime.
- **Kernel-Level Interventions:** The device's serial number, brand/model information, and other hardware identifiers are changed at the kernel level, simulating the device as a "real physical phone."

5. Technical Requirements List

Feature	Requirement	Notes
Server Type	Oracle Cloud Ampere A1 / AWS Graviton	Native ARM64 Processor
Operating System	Ubuntu 22.04 LTS	With Kernel 5.15+ support
Memory (RAM)	Minimum 16GB	Critical for stability and speed
Storage	50GB+ NVMe SSD	For container images and log management